

Applying Probabilistic Forecasting of Multidimensional Time Series to Electricity Load Analysis

Wenxiang Yin, Wen Ye, Guang Liu

School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876, P. R. China

E-mail: {yinwenxiang, yewen, liuguang}@bupt.edu.cn

Abstract: Electricity load is a crucial time series for electricity power systems, and its analysis and forecasting are essential for making informed decisions regarding future power infrastructure construction and power generation. Current deterministic point forecasting methods for electricity load forecasting have limitations. Probabilistic forecasting, on the other hand, provides more comprehensive analysis information by offering the interval and probability distribution of future changes. However, existing probabilistic schemes for time series forecasting, such as Variational Auto-Encoder (VAE), suffer from time-consuming training, large volume of model parameters, and inability to capture long-term dependencies in series. To address this challenge, this paper proposes a new probabilistic forecasting model V-trans that combines the self-attention mechanism and VAE. The model uses the self-attention scheme to capture long dependencies and VAE to extrapolate probability distribution in prediction results while minimizing computation costs and model storage. Evaluation experiments demonstrate that the proposed model has higher forecasting accuracy.

Key Words: Electricity Load Forecasting, Time Series, Probabilistic Forecasting, Variational Inference, self-attention

1 Introduction

Electricity load forecasting is to predict future electricity load based on relevant historical electricity load data of a region or building and other covariate data [1,2]. Two types of methods can be chosen for electricity load forecasting: traditional time series model and deep neural networks [3-5]. Although traditional time series models such as conventional regression-based methods still have a wide range of application scenarios [6,7], their weakness is that they require manual selection of some a priori knowledge such as trends and seasonality. Moreover, when the complexity of the research object is too high and the influence factors are too many, they have a poor prediction accuracy [8].

At present, deep neural networks have been widely used in various fields. There are also many application examples in the field of electricity load forecasting. For example, recurrent neural networks (RNN). However, RNNs are difficult to extract long dependencies because of the vanishing and the exploding gradient problems [9], which

are still difficult to solve despite the proposed techniques such as dropout [10], and improved models such as Long Short Term Memory (LSTM) and Gate Recurrent Unit (GRU) [11-13]. However, for applications such as electricity load forecasting, there are long- and short-term repetitive patterns. For example, daily and weekly, hourly and other patterns. Therefore, how to capture long-term dependencies becomes the key to achieve satisfactory forecasting results [14]. Transformer [15] model has achieved good results in numerous forecasting fields because of its self-attention mechanism [16]. The self-attention mechanism can capture both global and local associations in one step, and does not limit the capture of long-term dependencies by sequence length as in RNN-based networks.

The Variational Auto-Encoder (VAE) [17] is a type of generative models that can describe the latent space of observed data in terms of probability distributions, and it already has relevant applications in time series. However, VAE models require a large amount of training data and time, as well as the inability to work with complex data

features [18,19].

This paper focuses on proposing the V-Trans model. Because of the strong correlation between electricity load data and weather data [20], the prediction accuracy can be effectively improved by adding covariates to input. By combining the VAE and self-attention mechanisms, the model has performed well in predicting long-term probability distributions.

2 Problem definition and preliminary

2.1 Problem definition

Suppose we have a collection of N univariate time series of electricity loads $\mathcal{Y} = \{y_{1:T}^i\}_{i=1}^N$ for different buildings, where $y_{1:T}^i = [y_1^i, y_2^i, \dots, y_T^i]$ and $y_t^i \in \mathbb{R}$ is the value of time series i at time t . Our target is to predict the values of the next t time steps of the N time series $\mathcal{Y}_{t+1:t+\tau}$. Besides, let $\mathcal{U} = \{u_{1:T}^i\}_{i=1}^N$ be a set of weather data in the vicinity of this building with dimension d related to the time series $\mathcal{Y}$, and some attributes of the building itself, which are called covariates of the time series and whose values are known. Thus, our target object is (1).

$$\begin{aligned} P(\mathcal{Y}_{t_0}^i | \mathcal{Y}, \mathcal{U}) &= p(y_{t_0+1:t_0+\tau}^i | y_{1:t_0}^i, u_{1:t_0+\tau}^i; \Phi) \\ &= \prod_{t=t_0+1}^{t_0+\tau} p(y_t^i | y_{1:t-1}^i, u_{1:t}^i; \Phi) \end{aligned} \quad (1)$$

To simplify the operation, we combine time series with covariates and concat them together to obtain the model input. This process is shown as (2) and (3), where the $\odot$ operator represents concatenation.

$$x_t^i \triangleq [y_{t-1}^i \odot u_t^i] \in \mathbb{R}^{d+1} \quad (2)$$

$$X^i = [x_1, \dots, x_T] \in \mathbb{R}^{T \times (d+1)} \quad (3)$$

In this way, the problem can be converted to $\mathcal{Y}_{t_0+1:t_0+\tau} \sim f(X_{1:t_0})$, and the optimization target becomes (4).

$$p(y_{i,t_0+1:t_0+\tau} | X_{1:t_0}; \Phi) \quad (4)$$

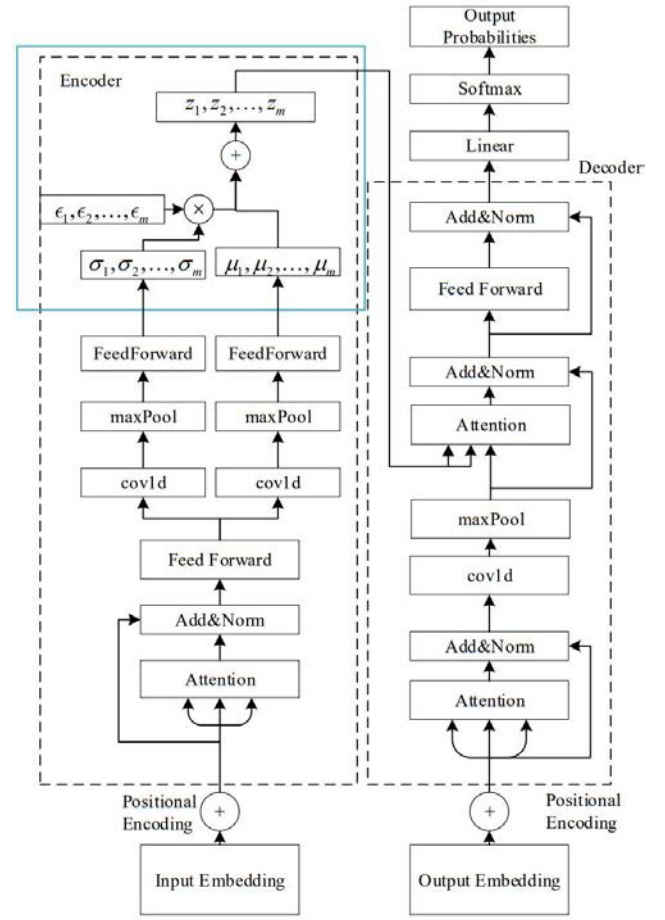


Fig. 1:V-trans model overview. Left: The encoder receives long time series sequences as input. The blue rectangle describes the reparameterization trick. Right: The decoder receives the same long time series sequences as input, but he will use a mask matrix to prevent future information leakage during training. The second self-attention layer will use the output of encoder as Query matrices and Key matrices.

2.2 Preliminary

Encoder-decoder Architecture Many generative models are divided into two parts: encoder and decoder. Where encoder converts the input X_i to a latent variable $\mathcal{Z}_i$ of fixed dimension, and then decoder gets the result $\mathcal{Y}_i$ based on the latent variable $\mathcal{Z}_i$. Our model involves “dynamic decoding”, where the decoder computes a new latent variable $\mathcal{Z}_i^{t+1}$ from $\mathcal{Z}_i^t$.

Reparameterization Trick The reparameterization trick is a method used in depth generative models to solve the problem of inability to derive derivatives when sampling from distributions with parameters.

$$L_\theta = \int p_\theta(z) f(z) dz \quad (5)$$

where $p_\theta(z)$ satisfy Gaussian distribution.

$$\begin{aligned} L_\theta &= \mathbb{E}_{z \sim \mathcal{N}(z; \mu, \sigma)}[f(z)] \\ &= \mathbb{E}_{\epsilon \sim q(\epsilon)}[f(g_\theta(\epsilon))] \\ &= \mathbb{E}_{\epsilon \sim \mathcal{N}(\epsilon; 0, 1)}[f(\epsilon \times \sigma + \mu)] \end{aligned} \quad (6)$$

By transferring the random quantities into a linear relationship, this solves the problem of non-transferable gradients due to sampling.

3 Model

In this section, the V-trans model is presented. It uses variational inference and self-attention to form encoder-decoder model to make probabilistic forecasting for long-term time series such as electricity load, and its model structure is illustrated in Fig. 1.

The encoder obtains temporal data features by embedding the input into position encoding and timestamp vectors. The self-attention mechanism is used to capture the internal correlation of the data. The encoder also uses convolutional operations to gain perception of local temporal features. Additionally, maximum pooling is used to simplify the model and improve the efficiency of utilizing data features. The encoder uses a feedforward neural network to fit the standard deviation of the latent variables, and uses reparameterization techniques and sampling to obtain the latent variable space. The resulting vectors are used as the key and value vectors for the decoder to obtain the correlation between historical values and future values.

The decoder consists of two self-attention layers. The first layer is used to obtain the feature autocorrelation matrix of the decoder input, which is a sequence embedding the encoder input and a placeholder timestamp for the prediction length. Through the self-attention mechanism, the matrix of future values applied to historical values is extracted. Through convolutional operations and maximum pooling, the output has the same features as the encoder, which is used as the query vector for the next self-attention layer. The key and value vectors are obtained by combining the outputs of the encoder with the query vector, to extract the dependency between future values and historical values. The decoder uses a residual network to increase its feature extraction ability. The model also includes normalization operations to eliminate the difference in magnitude between data.

3.1 Probabilistic forecasting based on variational inference

In probabilistic forecasting models, we assume that the true distribution has the result of a known combination of one or more probability distributions, and it is the parameters of the probability distribution that determine the sample distribution. Maximum likelihood estimation can calculate the parameters of the probability distribution when the posterior probabilities are simple. But the real world often makes it impossible to compute the integral because the posterior probability form is too complex, and it is impossible to compute the corresponding parameters by the likelihood function. Therefore, we want to approximate it by a simple function. If we can find a lower bound for $f(x)$ and then optimize the lower bound, we achieve the approximation of $f(x)$, which is the variational inference.

The $X = \{x^i\}_{i=1}^N$ denotes the observed time series values, and the $Z = \{z^i\}_{i=1}^N$ denotes the latent variable used to generate the probability distribution of x . The θ denotes the parameters of the model. Thus, the goal of our entire model is to minimize (7) with (8).

$$D_{KL}(q(z|x) || p(z|x)) \quad (7)$$

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz \quad (8)$$

So, we deduce that (9).

$$p_\theta(z|x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(x)} \quad (9)$$

Nevertheless, $p_\theta(x)$ is not directly computable. Also, for the posterior $p_\theta(z|x)$ is intractable. We assume a distribution $q_\phi(z|x)$ to approximate the true posterior $p_\theta(z|x)$.

We obtain the approximate probability function of the distribution of N data points.

$$\begin{aligned} \log p_\theta(x^{(i)}) &= D_{KL}(q_\phi(z|x^{(i)}) || p_\theta(z|x^{(i)})) \\ &\quad + \mathcal{L}(\theta, \phi; x^{(i)}) \end{aligned} \quad (10)$$

$$\begin{aligned} \mathcal{L}(\theta, \phi; x^{(i)}) &= E_{q_\phi(z|x)}[-\log q_\phi(z|x) \\ &\quad + \log p_\theta(x, z)] \end{aligned} \quad (11)$$

According to (12), we deduce that (13).

$$D_{KL}(q_\phi(z|x^{(i)}) || p_\theta(z|x^{(i)})) > 0 \quad (12)$$

$$\log p_\theta(x^{(i)}) \geq \mathcal{L}(\theta, \phi; x^{(i)}) \quad (13)$$

$$\mathcal{L}(\theta, \phi; x^{(i)}) = E_{q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)} | z)] - D_{KL}(q_{\phi}(z | x^{(i)}) || p_{\theta}(z)) \quad (14)$$

Equation (14) is what we call evidence lower bound (ELBO). The larger the $\mathcal{L}(\theta, \phi; x^{(i)})$, the closer $q_{\phi}(z | x^{(i)})$ and $p_{\theta}(z)$ are, and our target becomes maximizing the function $\mathcal{L}(\theta, \phi; x^{(i)})$.

3.2 Extracting long-term dependencies using self-attention mechanisms

The self-attention mechanism helps the transformer model to capture long-term dependencies. This advantage makes the transformer model well suited for time series forecasting. We apply self-attention mechanism to our probabilistic forecasting model, which enables V-trans to perform better in long-term time series forecasting.

A self-attention module convert U into H different query matrices $Q_h = UW_h^Q$, key matrices $K_h = UW_h^K$, and value matrices $V_h = UW_h^V$ in parallel, with $h \in \{1, \dots, H\}$.

Here $W_h^Q, W_h^K \in \mathbb{R}^{(d+1) \times d_k}$ and $W_h^V \in \mathbb{R}^{(d+1) \times d_v}$ are parameters for linear transformation. The attention function is defined as (15).

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (15)$$

The multi-headed attention layer will concat these H different outputs together to obtain an $h \times d_v$ dimensional output, and then obtain the final value with linear transformation. $W^O \in \mathbb{R}^{hd_v \times (d+1)}$.

$$O_M = MultiHead(Q, K, V) = Concat(O_1, \dots, O_H)W^O \quad (16)$$

where

$$O_h = Attention(Q_h, K_h, V_h) \quad (17)$$

3.3 Avoiding posterior collapse

We use a variational autoencoder to learn the distribution of time series data and obtain probabilistic forecasting. We optimize the ELBO defined in (14) during training. However, during the training of a variational autoencoder, there is a challenge known as posterior collapse, where the decoder neglects the latent variables output by the encoder and only considers them as noise. In this case, the posterior distribution of the encoder will perfectly match the prior distribution because it does not need to transmit any

information to the decoder. Therefore, we choose a general approach of optimizing the β -VAE instead of optimizing (18).

$$\mathcal{L}(z, x) = \mathbb{E}_{q(z|x)} (\log(p(x | z))) - \beta D_{KL}(q(z | x) || p(z)) \quad (18)$$

To optimize the (18), we use the Adam optimizer. In order to further reduce the risk of posterior collapse, we gradually increase the hyperparameter β from zero at the beginning of training to its pre-set value. We have implemented an exponentially increasing β feature. In our experiments, we have set β to 0.5.

Implementation Details We implement our model using Pytorch and used two NVIDIA GPUs for acceleration. We have used a batch size of 64 and trained the model for 30 epochs. The multi-head self-attention mechanism is composed of 8 parallel self-attention mechanisms.

3.4 Model training

The model can be divided into two parts: the encoder and the decoder, which are connected through latent variables. Algorithm 1 and 2 describe the training processes for the encoder and the decoder, respectively.

Algorithm 1 : V-trans model encoder

Input: time series dataset Y , covariates dataset U

Output: latent vector z

1. Randomly initialize model parameters
 2. $X = concat(Y, U)$
 3. **while** 1 **do**
 4. $J \leftarrow$ Randomly select M points from set X
 5. $\epsilon \leftarrow$ Random noise generated from the standard normal distribution
 6. $g \leftarrow \nabla_{\theta, \phi} \mathcal{L}^M(\theta, \phi; X^M, \epsilon)$ (Gradients of mini-batch estimator)
 7. $J = concat(J, PE(pos, i))$
 8. $O_h^{en} = Attention(Q_h^{en}, K_h^{en}, V_h^{en})$
 9. $O^{en} = MLP(concat(O_1^{en} \dots O_H^{en}) + U)$
 10. $\mu = MLP(maxPool(cov1d(O^{en})))$
 11. $\sigma = MLP(maxPool(cov1d(O^{en})))$
 12. $z = \mu + \epsilon \times \sigma$
 13. Use g to update model parameters.
 14. **end**
-

Algorithm 2 : V-trans model decoder

Input: time series dataset Y , covariates dataset U ,
latent vector z

Output: Future time series probability distribution

1. Randomly initialize model parameters
 2. $X = \text{concat}(Y, U)$
 3. **while** 1 **do**
 15. $J \leftarrow$ Randomly select M points from set X
 4. $g \leftarrow \nabla_{\theta, \phi} \mathcal{L}^M(\theta, \phi; X^M, \epsilon)$ (Gradients of mini-batch estimator)
 5. $U = \text{concat}(U, PE(pos, i))$
 6. $Q_h^{de1} = ZW_h^Q, K_h^{de1} = ZW_h^K, V_h^{de1} = UW_h^K$
 7. $O_h^{de1} = \text{Attention}(Q_h^{de1}, K_h^{de1}, V_h^{de1})$
 8. $U^{de1} = \text{concat}(O_1^{de1} \dots O_H^{de1}) + U$
 9. $O^{de1} = \text{maxPool}(\text{covld}(U^{de1}))$
 10. $O_h^{de2} = \text{Attention}(Q_h^{de2}, K_h^{de2}, V_h^{de2})$
 11. $U^{de2} = \text{concat}(O_1^{de2} \dots O_H^{de2}) + O^{de1}$
 12. $O^{de2} = \text{maxPool}(\text{covld}(U^{de2}))$
 13. **end**
-

4 Experiment

4.1 Dataset

The experiment uses real electricity load data from a region of the United States throughout 2016 as a dataset, which is a million-scale dataset, containing 1448 different buildings. These buildings were used for a variety of purposes, including education, residential, entertainment/public assembly, office, parking, retail, public service, warehouse, factory, medical service, and other types of buildings. The dataset covers the time range from January 1, 2016 to December 31, 2016, which has a one-hour sampling interval. The dataset includes daily electricity load data, along with weather data from the nearest weather station to the building, including air temperature (degrees Celsius), cloud cover, one-hour rainfall (millimetres), sea level pressure (hundred Pascal), wind direction (compass direction 0-360), and wind speed (meters per second).

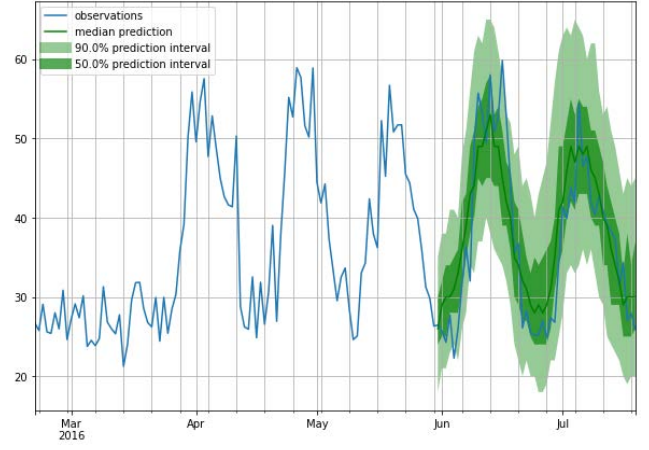


Fig. 2: The x-axis of the image represents the timestamp of the time series, while the y-axis represents the value of the time series. The blue line represents the observed time series data. The green line is the average prediction made by the model. The green region represents the confidence interval that we have set. The depth of the region distinguishes different confidence intervals.

4.2 Evaluation metrics

Mean Absolute Error (MAE) and Mean Square Error (MSE) [21] are selected as the evaluation metrics, which are defined in (19) and (20) as shown below. ρ -MAE is selected as the evaluation metrics for measuring the probabilistic forecasting, and $\rho_{i,j}$ represents the j -th percentile of the predicted probability distribution of the model.

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |(y_i - \hat{y}_i)| \quad (19)$$

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \quad (20)$$

$$\rho\text{-MAE} = \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^{99} |(y_i - \rho_{i,j})| \quad (21)$$

4.3 Result and analysis

Baselines We have selected four time-series forecasting methods for comparison, including DeepAR [22] (Salinas D, Flunkert V, Gasthaus J 2017), LSTM [14] (Hochreiter and Schmidhuber 1997), VAE [17] (Diederik P. Kingma and Max Welling 2013), Transformer [15] (A Vaswani and N Shazeer 2017).

As shown in Fig. 2, the model predicts the corresponding confidence interval based on the specified confidence level, and can also set multiple confidence levels to obtain multiple prediction intervals. The appropriate confidence

Table 2 Comparison of experimental results for univariate electricity load forecasting

Prediction length		180			360		
Encoder's input		180	360	720	360	720	1440
LSTM	MSE	2.035	1.950	1.832	3.303	3.197	3.029
	MAE	3.542	2.829	2.755	4.620	3.404	3.395
Transformer	MSE	1.794	1.524	1.467	1.897	1.774	1.632
	MAE	2.819	2.467	2.317	2.229	2.390	2.013
DeepAR	MSE	2.431	2.342	1.983	3.436	3.047	2.688
	MAE	3.903	3.062	2.736	4.892	4.484	3.423
VAE	MSE	4.004	3.679	3.118	4.422	3.963	3.828
	MAE	7.994	5.835	4.931	8.756	6.496	5.043
V-trans	MSE	1.876	1.487	1.323	2.126	1.662	1.438
	MAE	3.311	2.626	2.284	2.340	2.114	1.939

Table 2: Comparison of experimental results of multivariate electricity load forecasting

Prediction length		180		
Encoder's input		180	360	720
Transformer	MSE	1.432	1.329	1.234
	MAE	1.891	1.763	1.450
VAE	MSE	3.328	3.023	2.671
	MAE	5.234	5.196	4.373
V-trans	MSE	1.523	1.433	1.149
	MAE	1.342	1.524	1.259

Table 3: $\rho - MAE$ metrics comparison for probabilistic forecasting models.

Prediction length		180		
Encoder's input		180	360	720
DeepAR	$\rho - MAE$	2.45	3.45	5.12
VAE	$\rho - MAE$	3.48	3.78	4.45
V-trans	$\rho - MAE$	1.81	1.92	2.4

level should be selected based on the actual situation. In our experiments, we have chosen confidence levels of 0.5 and 0.9.

Tables 1 and 2 summarize the univariate and multivariate evaluation metrics for all models. In the experiment the model were set up with different input lengths and prediction horizon.

Univariate Forecasting In the univariate model, forecasts are made based on electricity load data only. From Table 1, we observe that: (1) In the long prediction range, the model of V-trans decreases more quickly as the length of the input increases to evaluate the metrics. This indicates that V-trans has better feature extraction ability for long time series. (2) The overall loss of V-trans is lower compared with other

probabilistic forecasting models such as VAE and DeepAR, which indicates that our probabilistic forecasting model has better prediction ability, especially in long time series. (3) V-trans has a lower error compared to LSTM models, with 8% (at 180), 24% (at 360), and 26% (at 720) reduction. (4) The V-trans and VAE models have 54% (at 180), 60% (at 360), and 58% (at 720) reduction in MSE metrics, respectively. This indicates that our model, after incorporating the self-attention mechanism, has better prediction results compared to the traditional probabilistic forecasting models.

Multivariate Forecasting In multivariate electricity load forecasting, in addition to electricity load as input, we also selected air temperature, one-hour rainfall, wind speed, and building type as covariates, all as inputs to encoder. Because of the hardware limitation, we only did multivariate experiments with a prediction length of 180. From Table 2, we find that V-trans still has the best results in multivariate forecasting, with a 7% reduction in MSE metrics compared to Transformer and a 57% reduction in VAE compared to VAE for long-term forecasting (at 720).

The difference between point forecasting and probability distribution forecasting is that point forecasting only needs to predict a precise point, while probability distribution prediction needs to predict the probability distribution of future data. For probability distribution forecasting models, simple measures such as MSE and MAE do not effectively measure the effectiveness of the model. Therefore, further comparison experiments need to be conducted using other indicators that can measure the characteristics of probability distribution.

As shown in Table 3, it can be observed that the V-trans model performs better in probabilistic forecasting

evaluation metrics. The probability distribution predicted by the V-trans probabilistic forecasting model has a more obvious peak feature. When the same confidence interval range is considered, the interval range is smaller, indicating that the V-trans model has better probability distribution characteristics.

5. Conclusion

This paper designs a new time series probabilistic forecasting model, V-trans, which achieves more accurate forecasting results in a real electricity load dataset compared to other probabilistic forecasting ones. Moreover, its probabilistic forecasting results are more detailed and explainable than that of mainstream point forecasting schemes such as Transformer.

References

- [1] Isaac Kofi Nti, Moses Teimeh, Owusu Nyarko-Boateng, and Adebayo Felix Adekoya. Electricity load forecasting: a systematic review. *Journal of Electrical Systems and Information Technology*, 7(1):1–19, 2020.
- [2] Liang Zhang, Jin Wen, Yanfei Li, Jianli Chen, Yunyang Ye, Yangyang Fu, and William Livingood. A review of machine learning in building load prediction. *Applied Energy*, 285:116452, 2021.
- [3] Zaiyong Tang, Chrys De Almeida, and Paul A Fishwick. Time series forecasting using neural networks vs. box-jenkins methodology. *Simulation*, 57(5):303–310, 1991.
- [4] Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: a survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.
- [5] Baran Yildiz, Jose I Bilbao, and Alistair B Sproul. A review and analysis of regression and machine learning models on commercial building electricity load forecasting. *Renewable and Sustainable Energy Reviews*, 73:1104–1122, 2017.
- [6] Grzegorz Dudek. Pattern-based local linear regression models for shortterm load forecasting. *Electric power systems research*, 130:139–147, 2016.
- [7] Sean J Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.
- [8] HS Hippert, DW Bunn, and RC Souza. Large neural networks for electricity load forecasting: Are they overfitted? *International Journal of forecasting*, 21(3):425–434, 2005.
- [9] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. Pmlr, 2013.
- [10] Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing coadaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.
- [11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [12] Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, and Jürgen Schmidhuber. Lstm: A search space odyssey. *IEEE transactions on neural networks and learning systems*, 28(10):2222–2232, 2016.
- [13] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [14] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long-and short-term temporal patterns with deep neural networks. In *The 41st international ACM SIGIR conference on research & development in information retrieval*, pages 95–104, 2018.
- [15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [16] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyou Zhou, Wenhui Chen, Yuxiang Wang, and Xifeng Yan. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in neural information processing systems*, 32, 2019.
- [17] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [18] Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. Deep variational information bottleneck. *arXiv preprint arXiv:1612.00410*, 2016.
- [19] Junxian He, Daniel Spokoyny, Graham Neubig, and Taylor BergKirkpatrick. Lagging inference networks and posterior collapse in variational autoencoders. *arXiv preprint arXiv:1901.05534*, 2019.
- [20] Vasudev Dehalwar, Akhtar Kalam, Mohan Lal Kolhe, and Aladin Zayegh. Electricity load forecasting for urban area using weather forecast information. In *2016 IEEE International Conference on Power and Renewable Energy (ICPRE)*, pages 355–359. IEEE, 2016.
- [21] Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International journal of forecasting*, 22(4):679–688, 2006.
- [22] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.